CMP2003 Rating Prediction
Group: Visionary Coders

Yavuz Selim Vurgun
Alp Emre Karaahmet
Emir Mert Saraç
Tutku Güneş

Introduction (Yavuz)

Our report includes two collaborative filtering approaches including UBCF (User-Based Collaborative Filtering) and IBCF (Item-Based Collaborative Filtering) to predict given test set users' ratings for given movies. We calculate similarities for items or users for the two algorithms and then predict ratings for users that for we do not know their ratings yet.

The code makes use of efficient data structures and precomputations of similarities to improve performance. We start by storing the input user-item-ratings for the training dataset, calculate similarities for items or users for the two approaches and use those precomputed similarities to predict a rating for the users in the test dataset. This report explains the data structures used, the approach to similarity calculation, and the workings of both UBCF and IBCF.

Storing the Input and Data Structures (Yavuz)

We start by parsing and storing the train dataset into an unordered_map of integers and unordered-maps which contain a user's ratings for a given movie. The use of unordered_maps enables us to access the elements quickly when we want to lookup a user's rating for a given movie.

We then store the test dataset which includes two integers representing userIds and movieIds. This is a simple structure to store all the test data in a vector of pairs in order. We don't need an unordered_map in this case since we are going through the list in an ordered way and thus we don't have to look up any value.

```
vector<pair<int, int>> testPairs; // Store userId, movieId pairs for test set
unordered_map<int, unordered_map<int, double>> trainData; // Store userId, movieId and rating for train set
unordered_map<int, unordered_map<int, double>> userSimilarities; // Store user-user similarities
unordered_map<int, unordered_map<int, double>> itemSimilarities; // Store item-item similarities

void store_data() {
    string line;
    bool isTest = false; // Determine if we're in the test dataset section

    // Storing the input
    while (getline(cin, line)) {
        if (line == "train dataset") {
            isTest = false;
            continue;
        }
        else if (line == "test dataset") {
            isTest = true;
            continue;
        }

        stringstream ss(line);
        int userId, movieId;
        double rating;

        // Adding the test set into the testPairs variable
        if (isTest) {
            ss >> userId >> movieId;
            testPairs.emplace_back(userId, movieId);
        }
        else { // Adding the training set into the trainData variable
            ss >> userId >> movieId >> rating;
            trainData[userId][movieId] = rating;
        }
    }
}
```

Similarity Calculation (UBCF User-User / IBCF Item-Item) (Alp)

For UBCF (User Based Collaborative Filtering), similarities between users are computed using cosine similarity. The function `calculateSimilarity` takes two users' rating maps as input and computes similarity with the given formula.

$$cosine(A, B) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i \times B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \times \sqrt{\sum_{i=1}^{n} B_i^2}}$$

```
double calculateSimilarity(const unordered_map<int, double>& user1, const unordered_map<int, double>& user2) {
    double dotProduct = 0, A_squared = 0, B_squared = 0;

    // Compute the dot product and the A_squared value of the first user's ratings
    for (const auto& item : user1) {
        int movieId = item.first;
        double rating1 = item.second;
        if (user2.count(movieId)) { // Check if the second user rated the same movie
            double rating2 = user2.at(movieId);
            dotProduct += rating1 * rating2;
        }
        A_squared += rating1 * rating1;
    }

    // Compute the B_squared of the second user's ratings
    for (const auto& item : user2) {
        double rating2 = item.second;
        B_squared += rating2 * rating2;
    }

    // Avoid division by zero
    if (A_squared == 0 || B_squared == 0) return 0.0;
    return dotProduct / (sqrt(A_squared) * sqrt(B_squared));
}
```

For IBCF, a similar function, `calculateItemSimilarity`, computes the cosine similarity between two items by considering the ratings given by all users. This function starts by transposing the training data to group ratings by items so that we can calculate the similarity in a similar fashion to the first function of calculating user similarity, enabling efficient similarity calculations. Precomputing the similarities and storing them in their respective data structures of unordered_maps of integers and unordered_maps of integers and doubles, indicating the user / movie IDs and their similarities enabling to lookup similarities when calculating the predictions using UBCF or IBCF methods. This saves us a big time of computing as we don't have the compute the similarities each time we need them.

```cpp
double calculateItemSimilarity(const unordered_map<int, double>& item1, const unordered_map<int, double>& item2) {
    double dotProduct = 0.0, A_squared = 0.0, B_squared = 0.0;

    for (const auto& item : item1) {
        int userId = item.first;
        double rating1 = item.second;
        if (item2.count(userId)) {
            double rating2 = item2.at(userId);
            dotProduct += rating1 * rating2;
        }
        A_squared += rating1 * rating1;
    }

    for (const auto& item : item2) {
        double rating2 = item.second;
        B_squared += rating2 * rating2;
    }

    if (A_squared == 0 || B_squared == 0) return 0.0;
    return dotProduct / (sqrt(A_squared) * sqrt(B_squared));
}
```

UBCF (User-Based Collaborative Filtering) (Emir)

In User-Based Collaborative Filtering, the goal is to predict a user's rating for an item by considering ratings from similar users. The function `predictRatingUBCF` takes the precomputed user similarities as input and sorts the similarities to find the top most similar users who have rated the same movie. A weighted average of these users' ratings is then computed, where the weights are their similarity scores.

```cpp
double predictRatingUBCF(int userId, int movieId, int k = 5) {
    vector<pair<double, int>> similarities; // Store similarity and user IDs

    // Use precomputed similarities
    for (const auto& [otherUserId, otherRatings] : trainData) {
        if (otherUserId != userId && otherRatings.count(movieId)) {
            double similarity = userSimilarities[userId][otherUserId];
            similarities.push_back({ similarity, otherUserId });
        }
    }

    // Sort the similarities in descending order
    sort(similarities.rbegin(), similarities.rend());

    double weightedSum = 0, similaritySum = 0;
    int count = 0;

    // Use the k most similar users to predict the rating
    for (const auto& sim : similarities) {
        if (count >= k) {
            break;
        } // Limit to k users
        double similarity = sim.first;
        int otherUserId = sim.second;
        double rating = trainData[otherUserId].at(movieId);
        weightedSum += similarity * rating; // Weighted sum of ratings
        similaritySum += abs(similarity);  // Sum of similarity weights
        count++;
    }

    // Return a default rating of 3.6 if no similar users are found
    if (similaritySum == 0) return 3.6;
    return weightedSum / similaritySum;
}
```

The precomputed user similarities, stored in `unordered_map<int, unordered_map<int, double>>`, enable quick retrieval of similarity values. Sorting and selecting the top neighbors ensure that only the most relevant users contribute to the prediction, improving accuracy, where we selected the number of neighbors (k) as 5 in our case. The method also includes an if clause to check if a retrieved similarity as 0, and returns a default rating of 3.6 to make sure there isn't a division by 0 in the code.

IBCF (Item-Based Collaborative Filtering) (Tutku)

Item-Based Collaborative Filtering predicts ratings by finding similarities between items instead of users. The function `predictRatingIBCF` uses precomputed item-item similarities, stored in an `unordered_map<int, unordered_map<int pop, double>>`. For a given user and target movie, the function identifies similar items that the user has already rated and computes a weighted average of those ratings.

Same with the UBCF function, precomputing the item similarities is really important for the time efficiency of our code, as we only compute the similarities once and then lookup from our data structure for the similarities whenever we need them. By using the cosine similarity metric and calculating the relationship between items with the function, we aim to predict the user's ratings for a given movie effectively. Like the UBCF function, our code has a check to avoid division by zero, in case we have a similaritySum value of 0, we return a default rating of 3.6.

```
double predictRatingIBCF(int userId, int movieId, int k = 5) {
    vector<pair<double, int>> similarities;

    for (const auto& [otherMovieId, similarity] : itemSimilarities[movieId]) {
        if (trainData[userId].count(otherMovieId)) {
            similarities.push_back({ similarity, otherMovieId });
        }
    }

    sort(similarities.rbegin(), similarities.rend());

    double weightedSum = 0.0, similaritySum = 0.0;
    int count = 0;

    for (const auto& sim : similarities) {
        if (count >= k) break;
        double similarity = sim.first;
        int similarMovieId = sim.second;
        double rating = trainData[userId].at(similarMovieId);
        weightedSum += similarity * rating;
        similaritySum += abs(similarity);
        count++;
    }

    if (similaritySum == 0) return 3.6;
    return weightedSum / similaritySum;
}
```